

5.2.2 LU factorization: The right-looking algorithm



In the launch of this week, we mentioned an algorithm that computes the LU factorization of a given matrix A so that

$$A = LU,$$

where L is a unit lower triangular matrix and U is an upper triangular matrix. We now derive that algorithm, which is often called the right-looking algorithm for computing the LU factorization.

Partition A , L , and U as follows:

$$A \rightarrow \begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix}, \quad L \rightarrow \begin{pmatrix} 1 & 0 \\ l_{21} & L_{22} \end{pmatrix}, \quad \text{and} \quad U \rightarrow \begin{pmatrix} v_{11} & u_{12}^T \\ 0 & U_{22} \end{pmatrix}.$$

Then $A = LU$ means that

$$\begin{pmatrix} \alpha_{11} & a_{12}^T \\ a_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ l_{21} & L_{22} \end{pmatrix} \begin{pmatrix} v_{11} & u_{12}^T \\ 0 & U_{22} \end{pmatrix} = \begin{pmatrix} v_{11} & u_{12}^T \\ l_{21}v_{11} & l_{21}u_{12}^T + L_{22}U_{22} \end{pmatrix}.$$

Hence

$$\begin{aligned} \alpha_{11} &= v_{11} & a_{12}^T &= u_{12}^T \\ a_{21} &= v_{11}l_{21} & A_{22} &= l_{21}u_{12}^T + L_{22}U_{22} \end{aligned}$$

or, equivalently,

$$\begin{aligned} \alpha_{11} &= v_{11} & a_{12}^T &= u_{12}^T \\ a_{21} &= v_{11}l_{21} & A_{22} - l_{21}u_{12}^T &= L_{22}U_{22}. \end{aligned}$$

If we overwrite the upper triangular part of A with U and the strictly lower triangular part of A with the strictly lower triangular part of L (since we know that its diagonal consists of ones), we deduce that we must perform the computations

- $a_{21} := l_{21} = a_{21}/\alpha_{11}$.
- $A_{22} := A_{22} - l_{21}a_{12}^T = A_{22} - a_{21}a_{12}^T$.
- Continue by computing the LU factorization of the updated A_{22} .

The resulting algorithm is given in [Figure 5.2.2.1](#).

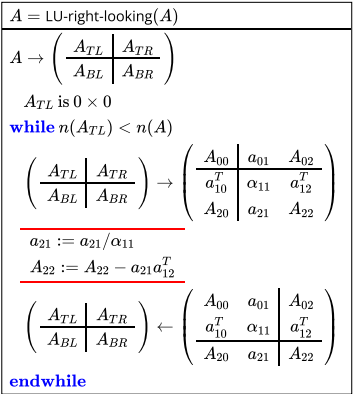


Figure 5.2.2.1. Right-looking LU factorization algorithm.

Before we discuss the cost of this algorithm, let us discuss a trick that is often used in the analysis of the cost of algorithms in linear algebra. We can approximate sums with integrals:

$$\sum_{k=0}^{n-1} k^p \approx \int_0^n x^p dx = \frac{1}{p+1} x^{p+1} \Big|_0^n = \frac{1}{p+1} n^{p+1}.$$

Homework 5.2.2.1. Give the approximate cost incurred by the algorithm in [Figure 5.2.2.1](#) when applied to an $n \times n$ matrix.

[Answer](#)
[Solution](#)

Consider the iteration where A_{TL} is (initially) $k \times k$. Then

- a_{21} is of size $n - k - 1$. Thus $a_{21} := a_{21}/\alpha_{11}$ is typically computed by first computing $1/\alpha_{11}$ and then $a_{21} := (1/\alpha_{11})a_{21}$, which requires $(n - k - 1)$ flops. (The cost of computing $1/\alpha_{11}$ is inconsequential when n is large, so it is usually ignored.)
- A_{22} is of size $(n - k - 1) \times (n - k - 1)$ and hence the rank-1 update $A_{22} := A_{22} - a_{21}a_{12}^T$ requires $2(n - k - 1)(n - k - 1)$ flops.

Now, the cost of updating a_{21} is small relative to that of the update of A_{22} and hence will be ignored. Thus, the total cost is given by, approximately,

$$\sum_{k=0}^{n-1} 2(n - k - 1)^2 \text{ flops.}$$

Let us now simplify this:

$$\begin{aligned} & \sum_{k=0}^{n-1} 2(n - k - 1)^2 \\ &= < \text{change of variable: } j = n - k - 1 > \\ & \sum_{j=0}^{n-1} 2j^2 \\ &= < \text{algebra} > \\ & 2 \sum_{j=0}^{n-1} j^2 \\ & \approx < \sum_{j=0}^{n-1} j^2 \approx \int_0^n x^2 dx = n^3/3 > \\ & \frac{2}{3} n^3 \end{aligned}$$

Approximately $\frac{2}{3}n^3$ flops.

Homework 5.2.2.2. Give the approximate cost incurred by the algorithm in [Figure 5.2.2.1](#) when applied to an $m \times n$ matrix.

▼ [Answer](#) ▼ [Solution](#)

Consider the iteration where A_{TL} is (initially) $k \times k$. Then

- a_{21} is of size $m - k - 1$. Thus $a_{21} := a_{21}/\alpha_{11}$ is typically computed by first computing $1/\alpha_{11}$ and then $a_{21} := (1/\alpha_{11})a_{21}$, which requires $(m - k - 1)$ flops. (The cost of computing $1/\alpha_{11}$ is inconsequential when m is large.)
- A_{22} is of size $(m - k - 1) \times (n - k - 1)$ and hence the rank-1 update $A_{22} := A_{22} - a_{21}a_{12}^T$ requires $2(m - k - 1)(n - k - 1)$ flops.

Now, the cost of updating a_{21} is small relative to that of the update of A_{22} and hence will be ignored. Thus, the total cost is given by, approximately,

$$\sum_{k=0}^{n-1} 2(m - k - 1)(n - k - 1) \text{ flops.}$$

Let us now simplify this:

$$\begin{aligned} & \sum_{k=0}^{n-1} 2(m - k - 1)(n - k - 1) \\ &= < \text{change of variable: } j = n - k - 1 > \\ & \sum_{j=0}^{n-1} 2(m - (n - j - 1) - 1)j \\ &= < \text{simplify} > \\ & \sum_{j=0}^{n-1} 2(m - n + j)j \\ &= < \text{algebra} > \\ & 2(m - n) \sum_{j=0}^{n-1} j + 2 \sum_{j=0}^{n-1} j^2 \\ & \approx < \sum_{j=0}^{n-1} j \approx n^2/2 \text{ and } \sum_{j=0}^{n-1} j^2 \approx n^3/3 > \\ & (m - n)n^2 + \frac{2}{3}n^3 \\ &= < \text{simplify} > \\ & mn^2 - \frac{1}{3}n^3 \end{aligned}$$

Approximately $mn^2 - \frac{1}{3}n^3$ flops.

Remark 5.2.2.2. In a practical application of LU factorization, it is uncommon to factor a non-square matrix. However, high-performance implementations of the LU factorization that use "blocked" algorithms perform a factorization of a rectangular submatrix of A , which is why we generalize beyond the square case.

Homework 5.2.2.3. It is a good idea to perform a "git pull" in the Assignments directory to update with the latest files before you start new programming assignments.

Implement the algorithm given in [Figure 5.2.2.1](#) as

```
function [ A_out ] = LU_right_looking( A )
```

by completing the code in [Assignments/Week05/matlab/LU_right_looking.m](#). Input is an $m \times n$ matrix A . Output is the matrix A that has been overwritten by the LU factorization. You may want to use [Assignments/Week05/matlab/test_LU_right_looking.m](#) to check your implementation.

► [Solution](#)